

INSTITUTE OF ENGINEERING
ADVANCED COLLEGE OF ENGINEERING AND MANAGEMENT
KALANKI, KATHMANDU
(AFFILIATED TO TRIBHUVAN UNIVERSITY)



ADVANCED COLLEGE
OF ENGINEERING AND MANAGEMENT

LAB REPORT

SUBJECT: Digital Signal Analysis and Processing
LAB NO: 3

Submitted By:

Name: Himal Joshi
Roll No: ACE079BCT028
Date: July 8, 2026

Submitted To:

**Department of Electronics and
Computer Engineering**

DISCRETE-TIME CONVOLUTION AND LTI SYSTEM PROPERTIES USING MATLAB

Objective

The objectives of this experiment are:

- Implement nested for loops by writing a custom discrete convolution engine. in MATLAB.
- Mathematically account for non-zero starting times and track time-axis boundaries in signal vectors.
- Validate fundamental Linear Time-Invariant (LTI) system properties (Commutative, Associative, and Distributive) empirically.
- Visualize how different signal interactions (pulses, exponentials, and steps) alter the output shape.
- Implement a loop-driven moving average filter to perform practical data-smoothing tasks over a noisy signal.

Theory

Introduction

Discrete-time convolution is one of the most fundamental operations in Digital Signal Processing (DSP). It describes how the output of a Linear Time-Invariant (LTI) system can be obtained by combining an input sequence with the system's impulse response. Convolution forms the mathematical foundation of digital filtering, communication systems, image processing, control systems and numerous signal-processing applications.

MATLAB provides an efficient environment for implementing convolution algorithms, visualizing discrete-time signals and verifying theoretical properties experimentally. In this experiment, convolution is implemented manually using nested loops and compared with MATLAB's built-in `conv()` function. Several signal interactions and important LTI system properties are also investigated.

Discrete-Time Convolution

For an input sequence $x[n]$ and impulse response $h[n]$, the discrete-time convolution is defined as

$$y[n] = x[n] * h[n]$$

or mathematically,

$$y[n] = \sum_{k=-\infty}^{\infty} x[k] h[n - k]$$

where

- $x[n]$ is the input sequence,
- $h[n]$ is the impulse response,
- $y[n]$ is the output sequence.

The convolution process consists of four basic operations:

1. Flip the impulse response in time.
2. Shift the flipped signal by the required index.
3. Multiply overlapping samples.
4. Sum all products to obtain one output sample.

For sequences of lengths L_x and L_h , the convolution output always has length

$$L_y = L_x + L_h - 1$$

Manual Convolution Algorithm

Although MATLAB provides the built-in function

`conv(x, h)`

it is important to understand how convolution is actually computed.

A manual convolution algorithm uses nested loops. The outer loop computes each output sample, while the inner loop performs multiplication and accumulation only when valid indices overlap. This implementation demonstrates the mathematical definition of convolution and provides insight into the convolution process.

Time-Axis Tracking

Unlike continuous signals, discrete-time sequences may start from arbitrary indices. Therefore, the output time axis must also be determined correctly.

If

$$x[n], \quad n = n_{x1} \dots n_{x2}$$

and

$$h[n], \quad n = n_{h1} \dots n_{h2}$$

then

$$n_{y,start} = n_{x1} + n_{h1}$$

$$n_{y,end} = n_{x2} + n_{h2}$$

The complete output index is therefore

$$n_y = n_{y,start} : n_{y,end}$$

Accurate time-axis tracking ensures that every sample corresponds to its correct discrete-time instant.

Linear Time-Invariant (LTI) Systems

A system is called Linear Time-Invariant when it satisfies two important properties.

Linearity

For any two signals,

$$x_1[n], x_2[n]$$

the system satisfies

$$T\{ax_1 + bx_2\} = aT\{x_1\} + bT\{x_2\}$$

where a and b are constants.

Time Invariance

If an input is delayed by n_0 samples,

$$x[n - n_0]$$

the output experiences the same delay without changing its shape.

Important Convolution Properties

Several mathematical properties make convolution useful in DSP.

Commutative Property

$$x * h = h * x$$

The order of convolution does not affect the output.

Associative Property

$$x * (h_1 * h_2) = (x * h_1) * h_2$$

Multiple systems may be connected in any sequence.

Distributive Property

$$x * (h_1 + h_2) = x * h_1 + x * h_2$$

Parallel systems can be analyzed independently before combining their outputs.

Impulse Response and Identity Property

The unit impulse sequence

$$\delta[n]$$

acts as the identity element of convolution.

$$x[n] * \delta[n] = x[n]$$

Similarly,

$$x[n] * \delta[n - k] = x[n - k]$$

which shifts the output without altering its shape.

Convolution of Rectangular Pulses

The convolution of two rectangular sequences illustrates how overlap between signals changes over time.

When both rectangles have equal widths, the output becomes a triangular waveform.

When one rectangle is shorter than the other, the output forms a trapezoidal waveform with a flat plateau.

This experiment demonstrates how convolution geometrically represents the overlap between two signals.

Step Response of First-Order Systems

A first-order discrete-time system with impulse response

$$h[n] = (0.7)^n u[n]$$

responds gradually to a unit step input

$$x[n] = u[n]$$

The convolution produces a smooth rising response that eventually approaches a steady-state value. Such responses are widely encountered in filters, control systems and electronic circuits.

Applications of Discrete-Time Convolution

Discrete-time convolution is extensively used in

- Digital filtering
- Audio and speech processing
- Image enhancement
- Communication systems
- Radar and sonar signal processing

- Biomedical signal analysis
- Control systems
- Machine learning signal preprocessing

MATLAB Codes

Task 1: Convolution using Loop vs Built-in Function

```

clc;
close all;
clear all;

x = [1 2 3 4];
h = [2 1 2];

Lx = length(x);
Lh = length(h);
Ly = Lx + Lh - 1;

y_manual = zeros(1,Ly);

for n = 1:Ly
    for k = 1:Lx
        j = n-k+1;
        if(j>=1 && j<=Lh)
            y_manual(n)=y_manual(n)+x(k)*h(j);
        end
    end
end

y_builtin = conv(x,h);

n = 0:Ly-1;

subplot(2,1,1)
stem(n,y_manual,'filled');
xlabel('n');
ylabel('Amplitude');
title('Manual Convolution / Himat Joshi, 28');

```

```

grid on;

subplot(2,1,2)
stem(n,y_builtin,'filled');
xlabel('n');
ylabel('Amplitude');
title('Built-in conv() Result / Himal Joshi, 28');
grid on;

```

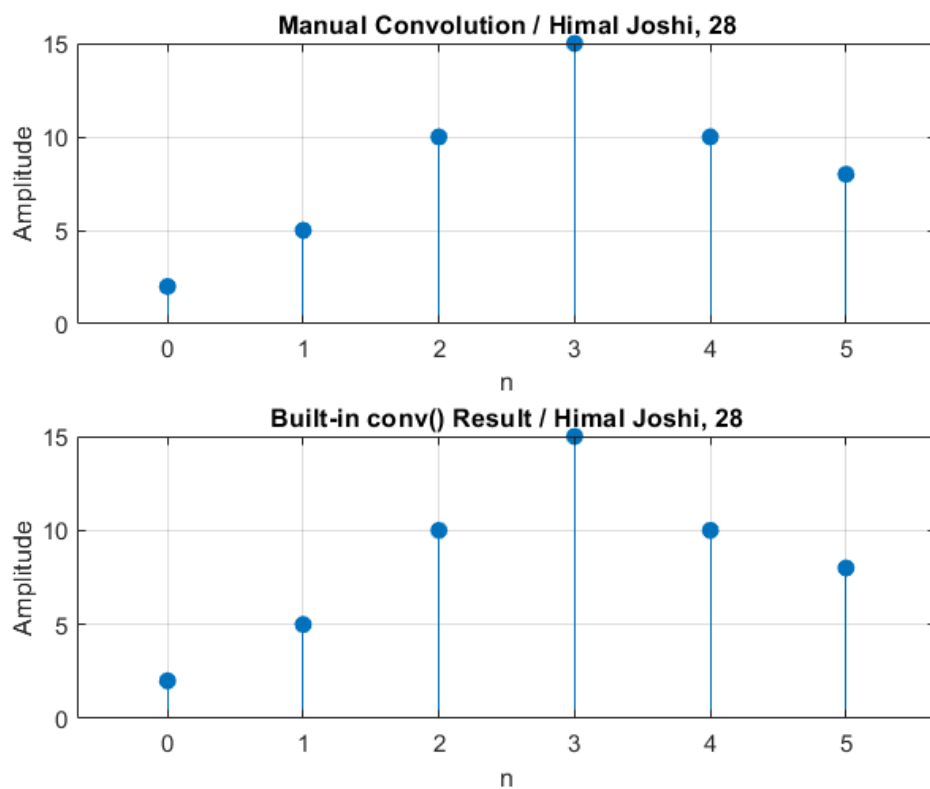


Figure 1: Manual Convolution and MATLAB Built-in Convolution

Task 2: Managing Shifted Time Origins

```
clc;
close all;
clear all;

x = [3 -1 2];
nx = -1:1;

h = [1 2 1 4];
nh = -2:1;

y = conv(x,h);

ny_start = nx(1)+nh(1);
ny_end = nx(end)+nh(end);
ny = ny_start:ny_end;

subplot(3,1,1)
stem(nx,x,'filled');
xlabel('n');
ylabel('Amplitude');
title('Input Signal x[n] / Himal Joshi, 28');
grid on;

subplot(3,1,2)
stem(nh,h,'filled');
xlabel('n');
ylabel('Amplitude');
title('Impulse Response h[n] / Himal Joshi, 28');
grid on;

subplot(3,1,3)
stem(ny,y,'filled');
xlabel('n');
ylabel('Amplitude');
title('Convolution Output y[n] / Himal Joshi, 28');
grid on;
```

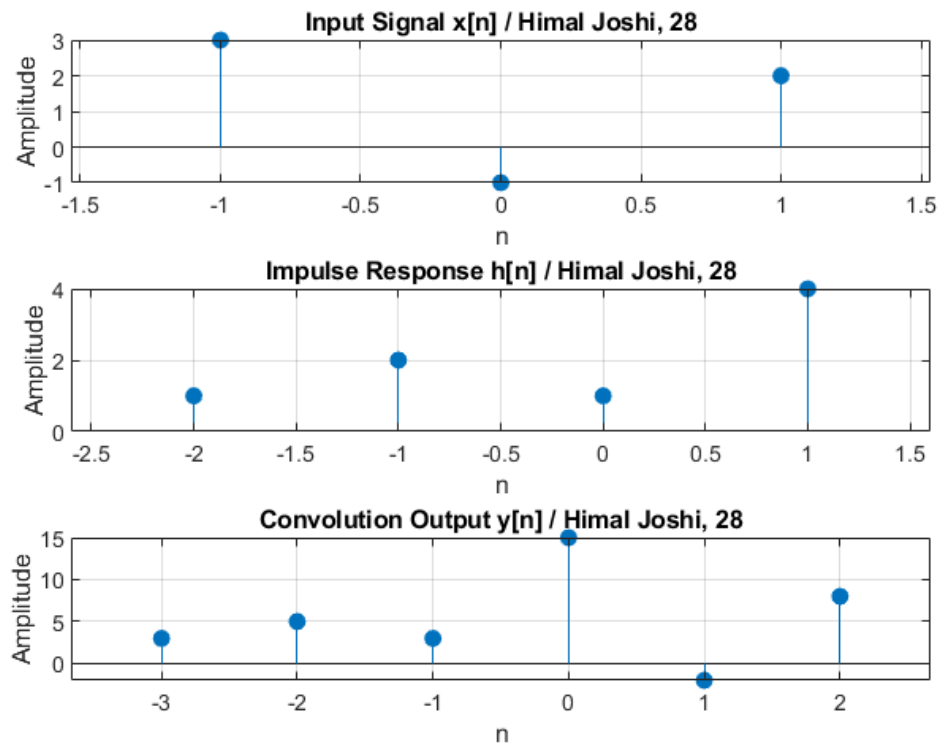


Figure 2: Convolution with Shifted Time Origins

Task 3: Verification of the Commutative Property

```
clc;
close all;
clear all;

x = [3 -1 2];
nx = -1:1;

h = [1 2 1 4];
nh = -2:1;

y1 = conv(x,h);
y2 = conv(h,x);

ny = (nx(1)+nh(1)):(nx(end)+nh(end));

if max(abs(y1-y2)) < 1e-12
    disp('Commutative Property Verified');
else
```

```

    disp('Commutative Property Failed');
end

stem(ny,y1,'o','LineWidth',1.5);
hold on;
stem(ny,y2,'x','LineWidth',1.5);

xlabel('n');
ylabel('Amplitude');
title('Commutative Property Verification / Himal Joshi, 28');
legend('x*h','h*x');
grid on;

```

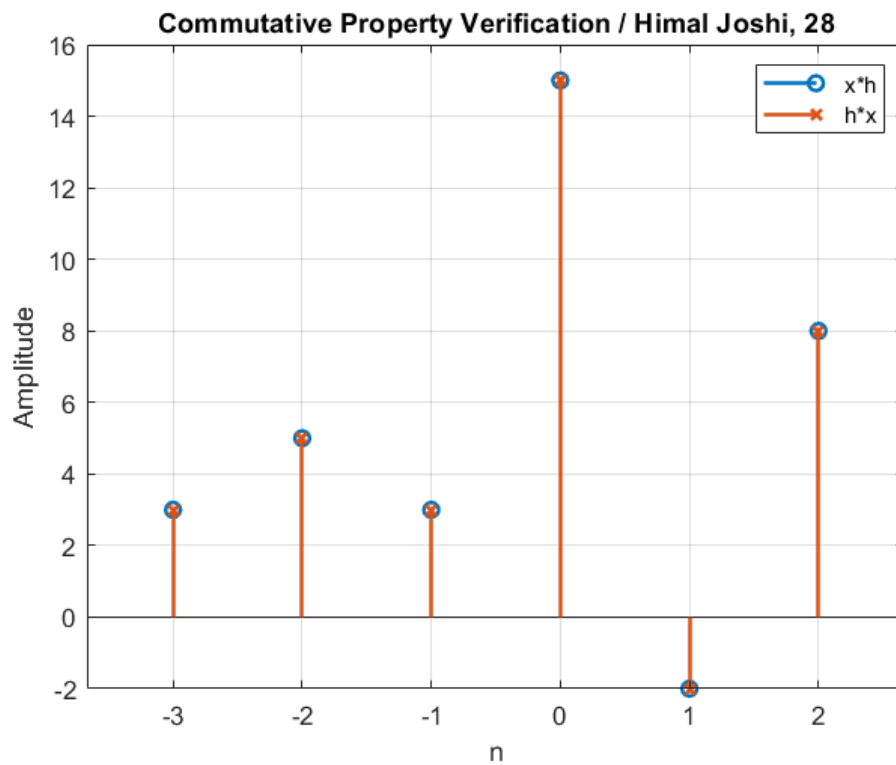


Figure 3: Verification of the Commutative Property

Task 4: Identity and Shifting Property of the Impulse

```
clc;
close all;
clear all;

h = [0.5 1 -0.5 2];
nh = 0:3;

delta = [1];
nd = 0;

delta_shift = [0 0 0 1];
nd2 = 0:3;

y1 = conv(h,delta);
ny1 = nh(1)+nd : nh(end)+nd;

y2 = conv(h,delta_shift);
ny2 = nh(1)+nd2(1):nh(end)+nd2(end);

subplot(3,1,1)
stem(nh,h,'filled');
xlabel('n');
ylabel('Amplitude');
title('Impulse Response h[n] / Himal Joshi, 28');
grid on;

subplot(3,1,2)
stem(ny1,y1,'filled');
xlabel('n');
ylabel('Amplitude');
title('h[n] * delta[n] / Himal Joshi, 28');
grid on;

subplot(3,1,3)
stem(ny2,y2,'filled');
xlabel('n');
ylabel('Amplitude');
```

```

title('h[n] * delta[n-3] / Himal Joshi, 28');
grid on;

```

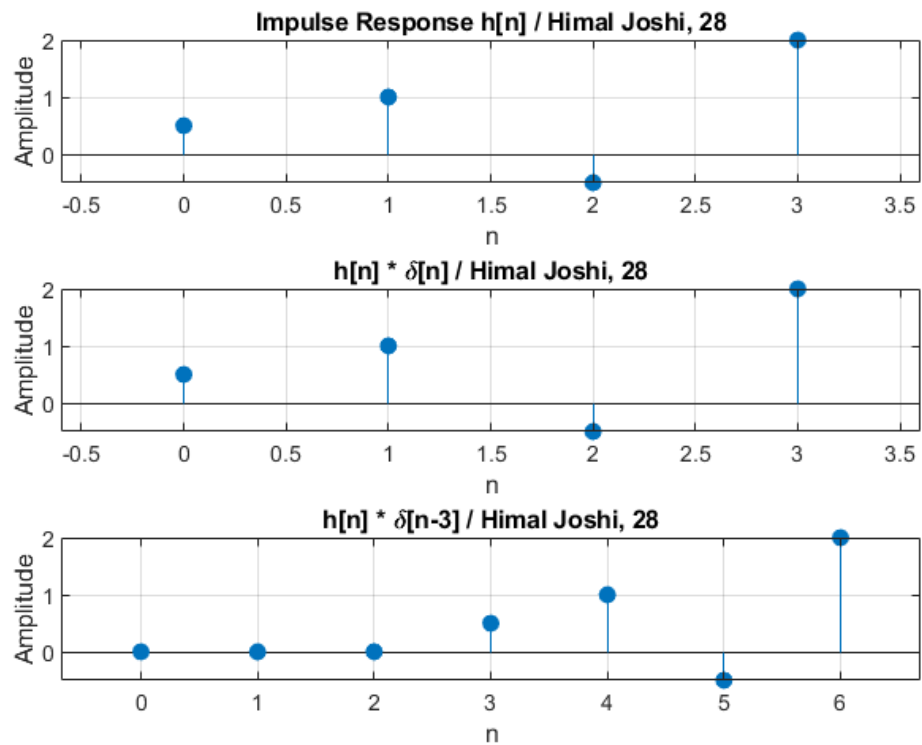


Figure 4: Identity and Shifting Property of the Unit Impulse

Task 5: Rectangle Meeting Rectangle

```

clc;
close all;
clear all;

x = ones(1,6);
h1 = ones(1,6);
h2 = ones(1,3);

y1 = conv(x,h1);
y2 = conv(x,h2);

subplot(2,2,1)
stem(0:5,x,'filled');
title('x[n] / Himal Joshi, 28');
grid on;

```

```

subplot(2,2,2)
stem(0:5,h1,'filled');
title('h[n] Length 6');
grid on;

subplot(2,2,3)
stem(0:length(y1)-1,y1,'filled');
title('Triangle Output');
grid on;

subplot(2,2,4)
stem(0:length(y2)-1,y2,'filled');
title('Trapezoid Output');
grid on;

```

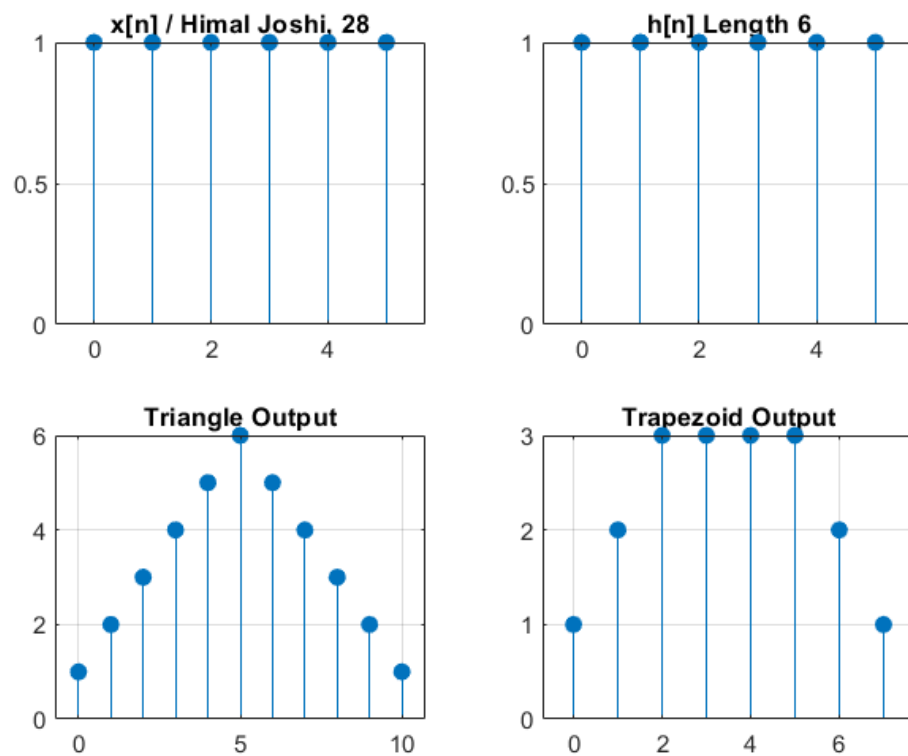


Figure 5: Convolution of Rectangular Pulses Showing Triangular and Trapezoidal Outputs

Task 6: Step Response of a First-Order System

```
clc;
close all;
clear all;

n = 0:15;

x = ones(size(n));
h = (0.7).^n;

y = conv(x,h);

ny = 0:length(y)-1;

subplot(3,1,1)
stem(n,x,'filled');
xlabel('n');
ylabel('Amplitude');
title('Unit Step Input / Himal Joshi, 28');
grid on;

subplot(3,1,2)
stem(n,h,'filled');
xlabel('n');
ylabel('Amplitude');
title('Impulse Response / Himal Joshi, 28');
grid on;

subplot(3,1,3)
stem(ny,y,'filled');
xlabel('n');
ylabel('Amplitude');
title('Step Response y[n] / Himal Joshi, 28');
grid on;
```

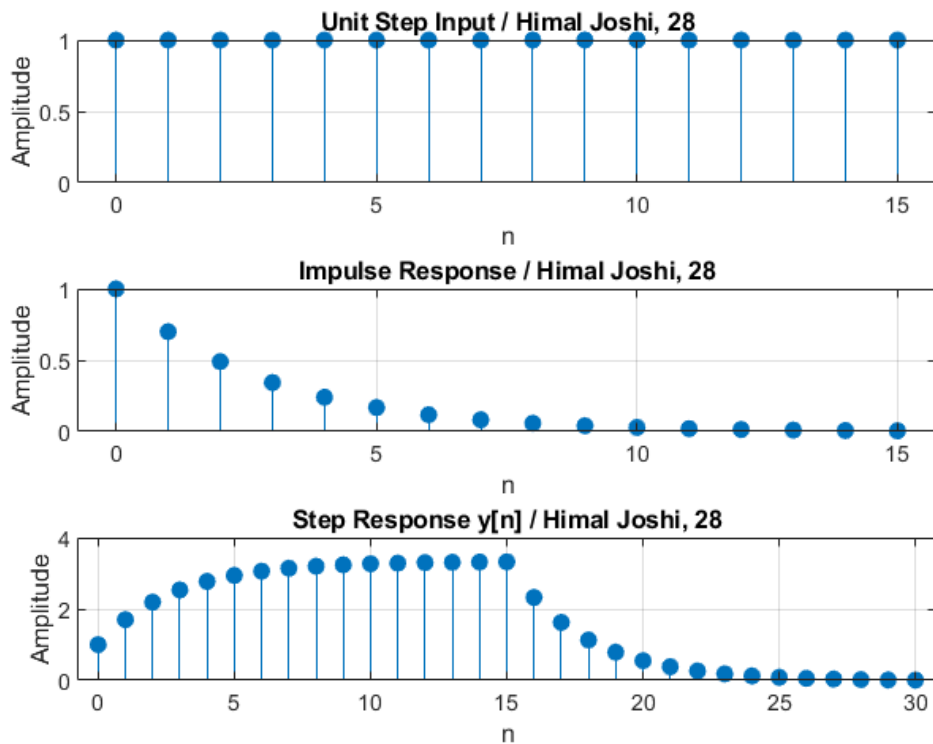


Figure 6: Step Response of a First-Order Discrete-Time System

Task 7: Verification of the Distributive Property

```
clc;
close all;
clear all;

x = [1 2 1];
h1 = [1 -1];
h2 = [2 1];

y_left = conv(x,h1+h2);
y_right = conv(x,h1)+conv(x,h2);

n = 0:length(y_left)-1;

if isequal(y_left,y_right)
    disp('Distributive Property Verified');
else
    disp('Distributive Property Failed');
end
```



```

stem(n,y_left,'o','LineWidth',1.5);
hold on;
stem(n,y_right,'x','LineWidth',1.5);

xlabel('n');
ylabel('Amplitude');
title('Distributive Property Verification / Himal Joshi, 28');
legend('LHS','RHS');
grid on;

```

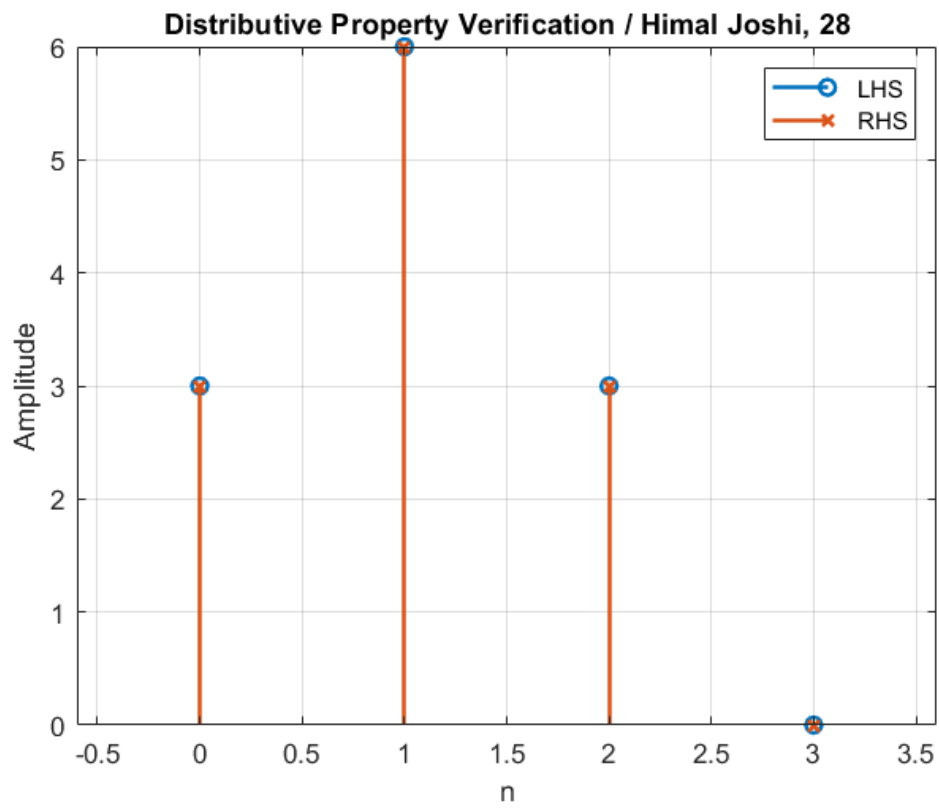


Figure 7: *Verification of the Distributive Property of Convolution*

Discussion

This experiment implemented discrete-time convolution and examined the properties that define LTI systems, using MATLAB. The manual convolution algorithm matched `conv()`'s output exactly, so the implementation checks out. Time-axis tracking for shifted signals behaved as expected, and the commutative and distributive properties held up numerically as well. The more useful part was working through the impulse response, the rectangular pulse convolution, and the step response of a first-order system that's where convolution stops being an abstract sum and starts looking like actual signal behavior. Across all of it, the results matched theory.

Conclusion

This experiment implemented discrete-time convolution and worked through the core properties of LTI systems in MATLAB. Checking the manual convolution algorithm against `conv()` confirmed it was correct, and the expected convolution properties held up under testing. The impulse response, rectangular pulse convolution, and first-order system response were the part that actually connected the math to how these systems behave in DSP. Between the theory and the MATLAB output lining up, the exercise did what it was meant to showing convolution as something you can compute, not just define.